10

Создание триггеров

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Рассматриваемые вопросы

- Различные типы триггеров
- Триггеры базы данных и их использование
- Создание триггеров базы данных
- Правила срабатывания триггеров базы данных
- Удаление триггеров базы данных

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Цели урока

В этом уроке вы научитесь создавать и использовать триггеры базы данных.

Типы триггеров

- **Триггер – это блок PL/SQL или процедура PL/SQL, связанные с таблицей, представлением, схемой или базой данных.**
- **Триггер срабатывает неявно всякий раз, когда происходит конкретное событие.**
- **Могут быть созданы:**
  - **Триггеры приложений, которые срабатывают, когда происходит событие в конкретном приложении**
  - **Триггеры базы данных, которые срабатывают, когда происходит событие с данными (например, DML-операция) или системное событие (например, соединение пользователя или останов БД) в схеме или базе данных.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Типы триггеров

Триггеры приложений (*application triggers*) выполняются неявно всякий раз, когда происходит событие, связанное с выполнением конкретной DML-операции внутри приложения. Например, такие триггеры широко используются в приложениях, разработанных с применением Oracle Forms Developer.

Триггеры базы данных (*database triggers*) выполняются неявно, когда происходят следующие события:

- DML-операции над таблицей;
- DML-операции над представлением (триггер `INSTEAD OF`);
- DDL-команды, например, `CREATE` и `ALTER`.

При этом неважно какой пользователь установил соединение или какое приложение используется. Триггеры базы данных также выполняются неявно, когда происходят некоторые действия пользователей или системные действия с базой данных, например, соединение пользователя или выполняемая АБД остановка базы данных

Примечание. Триггеры базы данных могут быть определены для таблиц или представлений. Когда операция DML выполняется над представлением, триггер `INSTEAD OF` задает действия, которые будут производиться. Если эти действия включают DML-операции над таблицами, тогда сработают триггеры, определенные для базовых таблиц.

Триггеры базы данных могут быть системными триггерами, заданными для базы данных или схемы. Триггеры, заданные для базы данных, срабатывают при каждом событии для всех пользователей, триггеры над схемой – для каждого события в схеме конкретного пользователя.

В этом уроке рассматривается создание триггеров базы данных. Триггеры для

системных событий обсуждаются в уроке “Дополнительные возможности триггеров”.

Проектирование триггеров: указания

- **Проектируйте триггеры для:**
  - **выполнения соответствующих действий**
  - **централизованного выполнения глобальных операций**
- **Не проектируйте триггеры,**
  - **если такие же действия уже встроены в сервер Oracle**
  - **которые повторяют другие триггеры**
- **Создавайте хранимые процедуры и вызывайте их из триггера, когда код PL/SQL слишком длинный.**
- **Чрезмерное использование триггеров может привести к появлению сложных взаимозависимостей, трудно сопровождаемых в больших приложениях.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Проектирование триггеров: указания

- Используйте триггеры, чтобы обеспечить выполнение конкретных действий для каждой конкретной операции.
- Используйте триггеры базы данных только для централизованных глобальных операций, которые должны быть выполнены для каждой команды, по которой срабатывает триггер, независимо от какого пользователя или приложения пришла эта команда.
- Не создавайте триггеры для дублирования или замещения функциональных возможностей, уже встроенных в базу данных Oracle. Например, не создавайте триггеры для введения ограничений, которые можно реализовать как декларативные ограничения. Реализация бизнес-правил обычно производится в следующем порядке:
  - Используйте ограничения, встроенные в сервер Oracle: главный ключ и т.д.
  - Разработайте триггеры базы данных или приложения (например, servlet или Enterprise JavaBean (EJB) на промежуточном уровне).
  - Используйте такие интерфейсы уровня представления, как Oracle Forms, HTML, Java ServerPages (JSP) и т.д., когда вы не можете реализовать бизнес правила способами, описанными выше. Такие правила относятся к правилам представления (presentation rule).
- Чрезмерное использование триггеров может привести к появлению сложных взаимозависимостей, которые могут быть трудно сопровождаемыми. Пользуйтесь триггерами только по необходимости и помните о рекурсивном и каскадном эффектах.
- Когда код PL/SQL, реализующий логику бизнес-правила, слишком длинный, создавайте хранимые процедуры или пакетные процедуры и вызывайте их из тела

триггера.

- Триггеры базы данных срабатывают для каждого пользователя всякий раз, когда происходит событие, для которого спроектирован триггер.

Создание DML-триггеров

Создание операторного или строчного триггера DML:

```
CREATE [OR REPLACE] TRIGGER имя_триггера
 время_срабатывания
 событие1 [OR событие2 OR событие3]
ON имя_объекта
[[REFERENCING OLD AS old | NEW AS new]
 FOR EACH ROW
 [WHEN (условие)]]
тело_триггера
```

- **Операторный триггер срабатывает один раз для команды DML.**
- **Строчный триггер срабатывает один раз для каждой обрабатываемой строки.**

Примечание: имена триггеров должны быть уникальными среди имен других триггеров той же самой схемы.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Создание DML-триггеров

Компоненты синтаксиса создания триггера:

- *имя_триггера* уникально идентифицирует триггер;
- *время_срабатывания* отражает, когда выполняется триггер: до (BEFORE), после (AFTER) или вместо (INSTEAD OF) события;
- событие определяет DML-операцию, вызывающую срабатывание триггера; значения: INSERT, UPDATE [OF столбец] или DELETE.
- *имя_объекта* (таблицы или представления), связанного с триггером.
- для *строчных триггеров* (*row triggers*) можно задать:
  - предложение REFERENCING, в котором указываются подходящие имена для ссылок на старые и новые значения текущей строки (по умолчанию OLD и NEW);
  - FOR EACH ROW определяет, что триггер строчный;
  - предложение WHEN, в котором в скобках задается условный предикат, проверяемый для каждой строки и определяющий необходимость выполнения тела триггера;
- *тело_триггера* – действие, выполняемое триггером, реализуемое одним из следующих способов:
  - анонимный блок, начинающийся с DECLARE или BEGIN и заканчивающийся END;
  - предложение CALL для вызова автономной или хранимой пакетной процедуры, например:
    CALL my_procedure;

Типы DML-триггеров

Тип триггера определяет, должно ли тело триггера выполняться для каждой строки или один раз для команды, по которой срабатывает триггер.

- **Операторный триггер:**
  - **выполняется только один раз для каждого события, по которому срабатывает триггер**
  - **тип триггера по умолчанию**
  - **срабатывает только один раз, даже если ни одна строка не затрагивается операцией DML**
- **Строчный триггер:**
  - **выполняется только раз для каждой строки, затрагиваемой данным событием**
  - **не выполняется, если ни одна строка не затрагивается операцией DML**
  - **определяется предложением `FOR EACH ROW`**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Типы DML-триггеров

Вы можете указать, что триггер будет выполняться один раз для каждой строки, затрагиваемой командой, по которой срабатывает триггер (например, многострочной командой `UPDATE`), или один раз для команды, независимо от числа затрагиваемых ею строк.

Операторный триггер

Операторный триггер (*statement trigger*) выполняется один раз для каждой команды, по которой он срабатывает, даже если ни одна строка не затронута.

Операторные триггеры полезны в случае, если действие триггера не зависит от данных в затрагиваемых строках или от данных, предоставляемых самим событием, по которому срабатывает триггер. В качестве примера можно привести триггер, выполняющий сложную проверку текущего пользователя в целях обеспечения безопасности.

Строчный триггер

Строчный триггер (*row trigger*) срабатывает каждый раз, когда в отношении таблицы возникает соответствующее событие. Если событие, вызывающее срабатывание триггера, не изменяет ни одной строки, строчный триггер вообще не выполняется.

Строчные триггеры полезны в случае, если действие триггера зависит от данных в затрагиваемых строках или от данных, предоставляемых самим событием, по которому срабатывает триггер.

Примечание: в строчных триггерах используются соответствующие имена для доступа к старым и новым значениям столбцов строки, обрабатываемой триггером.

Время срабатывания

Когда должен сработать триггер?

- **`BEFORE`: выполнять тело триггера до события, связанного с выполнением DML-операции над таблицей.**
- **`AFTER`: выполнять тело триггера после события, связанного с выполнением DML-операции над таблицей.**
- **`INSTEAD OF`: выполнять тело триггера вместо команды, вызывающей срабатывание триггера. Этот триггер используется для представлений, которые иным способом нельзя изменить.**

Примечание: несколько однотипных триггеров, определенных для одного и того объекта, срабатывают в произвольном порядке.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Время срабатывания

Триггеры **`BEFORE`** часто используются в следующих ситуациях:

- Необходимо выяснить, следует ли разрешить завершение команды, вызывающей срабатывание триггера. С помощью возбуждаемого в триггере исключения можно аннулировать ненужную обработку и отменить изменения.
- Требуется вычислить производные значения столбцов до завершения команды `INSERT` или `UPDATE`, по которой срабатывает триггер.
- Необходимо проинициализировать глобальные переменные или флаги, а также проверить сложные бизнес-правила.

Триггеры **`AFTER`** часто используются в следующих целях:

- Вы хотите завершить действие команды, прежде чем будет выполнена обработка, заданная в триггере.
- Если уже имеется триггер `BEFORE`, но триггер `AFTER` может выполнить какое-то другое действие по одной и той же команде DML.

Триггер **`INSTEAD OF`** предоставляет прозрачный способ внесения изменений в представление, в которое нельзя прямо внести изменения с помощью SQL DML. Триггер `INSTEAD OF` прорабатывает незаметно как фоновая операция, непосредственно выполняя над базовыми таблицами действия, закодированные в теле триггера.

Примечание. Последовательность срабатывания нескольких однотипных триггеров, определенных для одной таблицы, произвольна. Если требуется, чтобы триггеры одного и того же типа срабатывали в определенном порядке, необходимо объединить их в один триггер, вызывающий различные процедуры в определенном порядке.

Последовательность срабатывания триггеров

Когда обрабатывается единственная строка, триггеры таблицы срабатывают в следующей последовательности:

Команда DML

```
INSERT INTO departments
   (department_id,department_name, location_id)
VALUES (400, 'CONSULTING', 2400);
```

Действия триггеров

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION_ID
10	Administration	1700
20	Marketing	1800
30	Purchasing	1700
...		
400	CONSULTING	2400

→ Операторный триггер **BEFORE**

→ Строчный триггер **BEFORE**

→ Строчный триггер **AFTER**

→ Операторный триггер **AFTER**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Последовательность срабатывания триггеров

Выбор между операторным и строчным триггером зависит от того, когда должен срабатывать триггер, – один раз на каждую строку, затрагиваемую командой, по которой срабатывает триггер, или один раз на каждую такую команду, независимо от числа обрабатываемых командой строк.

Если команда DML , вызывающая срабатывание триггера, затрагивает только одну строку, то и операторный триггер, и строчный триггер сработают по одному разу.

Пример

При выполнении команды SQL операторный и строчный триггеры сильно не отличаются, поскольку только одна запись вставляется в таблицу по оператору `INSERT` с таким синтаксисом.

Последовательность срабатывания триггеров

Когда обрабатывается много строк, триггеры таблицы срабатывают в следующей последовательности:

```
UPDATE employees
  SET salary = salary * 1.1
  WHERE department_id = 30;
```

→ Операторный триггер BEFORE

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
114	Raphaely	30
115	Khoo	30
116	Baida	30
117	Tobias	30
118	Himuro	30
119	Colmenares	30

→ Строчный триггер BEFORE
→ Строчный триггер AFTER
...
→ Строчный триггер BEFORE
→ Строчный триггер AFTER
...
→ Операторный триггер AFTER

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Последовательность срабатывания триггеров (продолжение)

Если команда DML, по которой срабатывает триггер, затрагивает несколько строк, то операторный триггер выполняется только один раз, а строчный – столько раз, сколько строк обрабатывает данная команда.

Пример

По команде SQL, приведенной на слайде, строчный триггер срабатывает столько раз, сколько строк таблицы удовлетворяет условию в предложении `WHERE`, т.е. сколько служащих в отделе 30.

Типы событий и тело триггера

Событие триггера:

- **Определяет, какая команда DML вызвала выполнение триггера**
- **Типы:**
  - **`INSERT`**
  - **`UPDATE [OF столбец]`**
  - **`DELETE`**

Тело триггера:

- **Определяет, какие действия выполняются**
- **Это блок PL/SQL или вызов (`CALL`) процедуры**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Типы событий триггера

Событием, вызывающим срабатывание триггера (*triggering event*), может быть команда `INSERT`, `UPDATE` или `DELETE` для таблицы.

- Если таким событием является команда `UPDATE`, вы можете включить список столбцов, чтобы указать, какие столбцы должны быть изменены, чтобы триггер сработал. Задать список столбцов для команд `INSERT` и `DELETE` невозможно, т.к. эти команды затрагивают целые строки.

```
. . . UPDATE OF salary . . .
```

- Событие, по которому срабатывает триггер, может включать одну, две или все три операции DML.

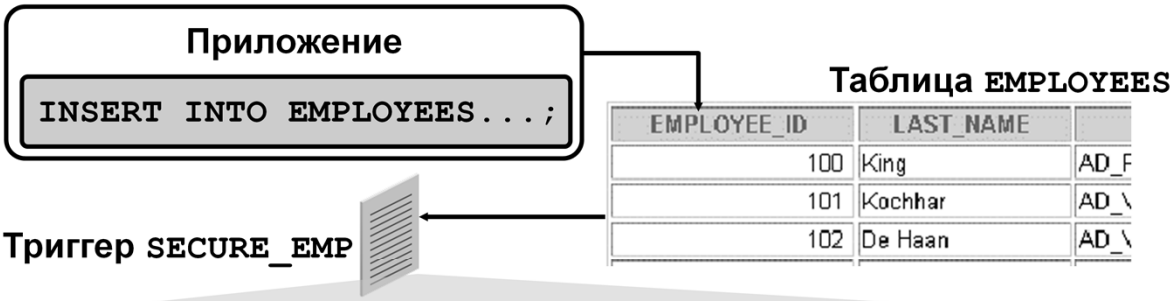
```
. . . INSERT or UPDATE or DELETE
. . . INSERT or UPDATE OF job_id . . .
```

Действие триггера (*trigger action*) – это то, что должно быть сделано в случае возникновения события, по которому срабатывает триггер. Тело триггера может содержать команды SQL и PL/SQL, а также определения конструкций PL/SQL, таких как переменные, курсоры, исключения и т.д. Вы также можете вызвать PL/SQL-процедуру или Java-процедуру.

Кроме того, с помощью соответствующих имен (квалификаторов) строчные триггеры могут обращаться к столбцам со старыми и новыми значениями строки, обрабатываемой триггером.

Примечание: размер триггера не может быть более 32 Кб.

Создания триггера, срабатывающего по команде DML




```
CREATE OR REPLACE TRIGGER secure_emp
BEFORE INSERT ON employees BEGIN
 IF (TO_CHAR(SYSDATE,'DY') IN ('SAT','SUN')) OR
     (TO_CHAR(SYSDATE,'HH24:MI')
              NOT BETWEEN '08:00' AND '18:00') THEN
  RAISE_APPLICATION_ERROR(-20500, 'You may insert'
    ||' into EMPLOYEES table only during '
    ||' business hours.');
  END IF;
END;
```

Copyright © 2004, Oracle. All rights reserved.

Создания триггера, срабатывающего по команде DML

В примере приведен *операторный триггер* BEFORE (BEFORE *statement trigger*) SECURE_EMP, созданный для того, чтобы операция INSERT не выполнялась до конца, если нарушены какие-либо условия. В результате действия триггера вставки в таблицу EMPLOYEES возможны только в определенные рабочие часы с понедельника по пятницу.

Если, например, пользователь попытается произвести вставку в таблицу EMPLOYEES в субботу, тогда триггер завершится с ошибкой, вследствие чего произойдет откат команды, по которой он сработал. RAISE_APPLICATION_ERROR – это встроенная процедура на сервере, которая посылает сообщение об ошибке пользователю и аварийно прерывает выполнение блока PL/SQL.

Сервер Oracle автоматически производит откат команды, вызвавшей срабатывание триггера, когда по какой-то причине триггер базы данных не может быть выполнен до конца

.

Проверка триггера SECURE_EMP

```
INSERT INTO employees (employee_id, last_name,
       first_name, email, hire_date,
       job_id, salary, department_id)
VALUES (300, 'Smith', 'Rob', 'RSMITH', SYSDATE,
      'IT_PROG', 4500, 60);
```

```
INSERT INTO employees (employee_id, last_name, first_name, email,
     *
ERROR at line 1:
ORA-20500: You may insert into EMPLOYEES table only during business hours.
ORA-06512: at "PLSQL.SECURE_EMP", line 4
ORA-04088: error during execution of trigger 'PLSQL.SECURE_EMP'
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Проверка триггера SECURE_EMP

Вставка записи в таблицу EMPLOYEES в нерабочие часы приводит к ошибке.

Использование условных предикатов

```
CREATE OR REPLACE TRIGGER secure_emp BEFORE
INSERT OR UPDATE OR DELETE ON employees BEGIN
 IF (TO_CHAR(SYSDATE,'DY') IN ('SAT','SUN')) OR
   (TO_CHAR(SYSDATE,'HH24')
      NOT BETWEEN '08' AND '18') THEN
   IF DELETING THEN RAISE_APPLICATION_ERROR(
    -20502,'You may delete from EMPLOYEES table'||
           'only during business hours.');
   ELSIF INSERTING THEN RAISE_APPLICATION_ERROR(
    -20500,'You may insert into EMPLOYEES table'||
           'only during business hours.');
   ELSIF UPDATING('SALARY') THEN
    RAISE_APPLICATION_ERROR(-20503, 'You may '||
     'update SALARY only during business hours.');
   ELSE RAISE_APPLICATION_ERROR(-20504,'You may'||
     ' update EMPLOYEES table only during'||
     ' normal hours.');
   END IF;
 END IF;
END;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Сочетания событий, вызывающих срабатывание триггера

Использование в теле триггера условных предикатов `INSERTING`, `UPDATING` и `DELETING` позволяет объединить несколько событий, вызывающих срабатывание триггера, в одно.

Пример

Создание триггера, ограничивающего все DML-операции с таблицей `EMPLOYEES` определенными рабочими часами с понедельника по пятницу.

Создание строчного триггера, срабатывающего по команде DML

```
CREATE OR REPLACE TRIGGER restrict_salary
BEFORE INSERT OR UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
  IF NOT (:NEW.job_id IN ('AD_PRES', 'AD_VP'))
     AND :NEW.salary > 15000 THEN
    RAISE_APPLICATION_ERROR (-20202,
      'Employee cannot earn more than $15,000.');
  END IF;
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Создание строчного триггера, срабатывающего по команде DML

Строчный триггер BEFORE (BEFORE row *trigger*) можно создать для того, чтобы предотвратить успешное завершение операция, когда нарушаются определенное условие.

В примере на слайде создается триггер для проверки того, что только определенные служащие могут зарабатывать более 15000.

Если пользователь попытается выполнить следующее изменение, будет возбуждено сообщение об ошибке:

```
UPDATE employees
SET salary = 15500
WHERE last_name = 'Russell';
```

В триггере будет возбуждено следующее исключение :

```
UPDATE EMPLOYEES
   *
ERROR at line 1:
ORA-20202: Employee cannot earn more than $15,000.
ORA-06512: at "PLSQL.RESTRICT_SALARY", line 5
ORA-04088: error during execution of trigger
"PLSQL.RESTRICT_SALARY"
```

Использование квалификаторов OLD и NEW

```
CREATE OR REPLACE TRIGGER audit_emp_values
AFTER DELETE OR INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
  INSERT INTO audit_emp(user_name, time_stamp, id,
    old_last_name, new_last_name, old_title,
    new_title, old_salary, new_salary)
  VALUES (USER, SYSDATE, :OLD.employee_id,
    :OLD.last_name, :NEW.last_name, :OLD.job_id,
    :NEW.job_id, :OLD.salary, :NEW.salary);
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использование квалификаторов OLD и NEW

В строчном триггере в ссылках на значение столбца до и после изменения указываются в качестве префикса *квалификаторы* OLD *и* NEW (OLD *and* NEW *qualifier*).

Операция с данными	Старое (OLD) значение	Новое (NEW) значение
INSERT	NULL	Вставляемое значение
UPDATE	Значение до изменения	Значение после изменения
DELETE	Значение до удаления	NULL

Замечания по использованию:

- Квалификаторы OLD и NEW доступны только в строчных триггерах (типа ROW).
- Перед этими квалификаторами необходимо ставить двоеточие (:) в каждой команде SQL и PL/SQL .
- В предложении WHEN, содержащем ограничивающее условие, двоеточие (:) перед квалификатором не указывается.

Примечание: строчные триггеры могут снизить производительность, если выполняется много изменений в больших таблицах.

Квалификаторы OLD и NEW: пример использования audit_emp

```
INSERT INTO employees
 (employee_id, last_name, job_id, salary, ...)
VALUES (999, 'Temp emp', 'SA_REP', 6000,...);

UPDATE employees
 SET salary = 7000, last_name = 'Smith'
 WHERE employee_id = 999;
```

```
SELECT user_name, timestamp, ...
FROM audit_emp;
```

USER_NAME	TIMESTAMP	ID	OLD_LAST_N	NEW_LAST_N	OLD_TITLE	NEW_TITLE	OLD_SALARY	NEW_SALARY
PLSQL	28-SEP-01			Temp emp		SA_REP		1000
PLSQL	28-SEP-01	999	Temp emp	Smith	SA_REP	SA_REP	1000	2000

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Квалификаторы OLD и NEW: пример использования audit_emp

Для таблицы EMPLOYEES создается триггер, который добавляет записи в пользовательскую таблицу AUDIT_EMP. Эта таблица содержит строки об изменениях, вносимых пользователями в таблицу EMPLOYEES. Триггер записывает значения некоторых столбцов до и после их изменения, используя в названиях столбцов квалификаторы OLD и NEW.

На слайде не показан столбец COMMENTS, входящий в таблицу AUDIT_EMP.

Ограничение действия строчного триггера: пример

```
CREATE OR REPLACE TRIGGER derive_commission_pct
BEFORE INSERT OR UPDATE OF salary ON employees
FOR EACH ROW
WHEN (NEW.job_id = 'SA_REP')
BEGIN
 IF INSERTING THEN
   :NEW.commission_pct := 0;
 ELSIF :OLD.commission_pct IS NULL THEN
   :NEW.commission_pct := 0;
 ELSE
   :NEW.commission_pct := :OLD.commission_pct+0.05;
 END IF;
END;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Ограничение действия строчного триггера: пример

Для ограничения действия триггера строками, удовлетворяющими определенным критериям, используется предложение WHEN.

В примере на слайде над таблицей EMPLOYEES создается триггер для вычисления размера комиссионных в случае, если в таблицу EMPLOYEES вставляется строка или изменяется оклад служащего.

Перед квалификатором NEW в предложении WHEN двоеточие не ставится, поскольку предложение WHEN находится вне блока PL/SQL.

Обобщенная модель выполнения триггеров

1. Выполнение всех операторных триггеров BEFORE.
2. В цикле для каждой затрагиваемой строки :
   a. Выполнение всех строчных триггеров BEFORE.
   b. Выполнение команды DML и проверка ограничений целостности.
   c. Выполнение всех строчных триггеров AFTER.
3. Выполнение всех операторных триггеров AFTER.

Примечание: проверка ограничений целостности может быть отложена до выполнения операции COMMIT.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Модель выполнения триггеров

По одной команде DML могут выполняться до четырех типов триггеров:

- операторные триггеры BEFORE и AFTER;
- строчные триггеры BEFORE и AFTER.

Событие или команда, по которой срабатывает триггер, могут вызвать проверку соблюдения одного или нескольких правил целостности. Однако проверку ограничений целостности можно отложить до выполнения операции COMMIT.

Триггеры также могут вызвать каскадное срабатывание других триггеров (*cascading triggers*).

Все действия и проверки, выполняемые в результате выполнения исходной команды SQL, должны быть успешно завершены. Если внутри триггера возникает сбой и соответствующее исключение не обрабатывается явно, все действия, выполненные в связи с исходной SQL-командой, отменяются, включая действия, выполненные сработавшими триггерами. Таким образом, правила целостности никогда не могут быть нарушены из-за триггеров.

В момент, когда срабатывает триггер, таблицы, на которые он ссылается, могут подвергаться изменениям в транзакциях других пользователей. В любом случае согласованное состояние всех измененных значений, которые триггер должен считывать (при запросе) или записывать (при обновлении) гарантируется.

Поддержка ограничения целостности с помощью триггера

```
UPDATE employees SET department_id = 999
 WHERE employee_id = 170;
-- Integrity constraint violation error
```

```
CREATE OR REPLACE TRIGGER employee_dept_fk_trg
AFTER UPDATE OF department_id
ON employees FOR EACH ROW
BEGIN
 INSERT INTO departments VALUES(:new.department_id,
          'Dept '||:new.department_id, NULL, NULL);
EXCEPTION
   WHEN DUP_VAL_ON_INDEX THEN
    NULL; -- не показывает исключение, когда такой
END;      -- отдел существует
/
```

```
UPDATE employees SET department_id = 999
 WHERE employee_id = 170;
-- Successful after trigger is fired
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Поддержка ограничения целостности с помощью триггера

Пример на слайде объясняет ситуацию, в которой ограничение целостности можно соблюсти с помощью триггера. Таблица `EMPLOYEES` имеет ограничение внешнего ключа, которое ссылается на столбец `DEPARTMENT_ID` таблицы `DEPARTMENTS`.

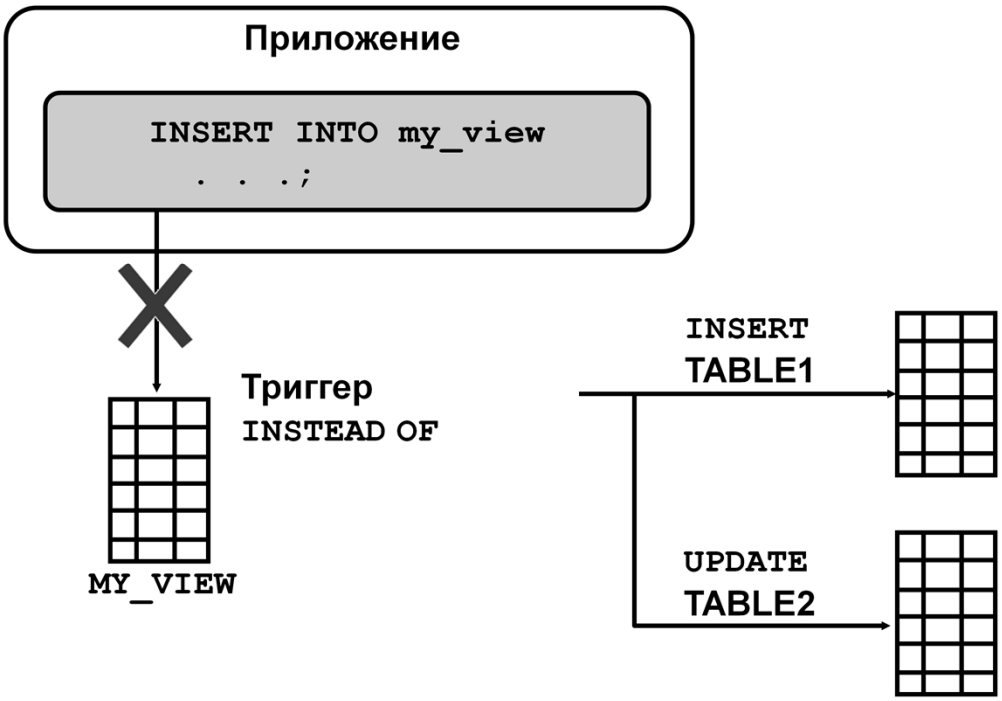
В первой команде SQL столбец `DEPARTMENT_ID` сотрудника с номером `EMPLOYEE_ID`, равным 170, меняется на 999.

Поскольку такого отдела нет в таблице `DEPARTMENTS`, возбуждается исключение – 2292, сообщающее о нарушении ограничения целостности.

Триггер `EMPLOYEE_DEPT_FK_TRG` создается для того, чтобы добавить новый отдел 999 в таблицу `DEPARTMENTS` (с помощью `:NEW.DEPARTMENT_ID`). При выполнении команды `UPDATE`, которая изменяет отдел сотрудника 170 на 999, срабатывает триггер. Затем проверяется ограничение. Так как триггер вставляет отдел 999 в таблицу `DEPARTMENTS`, проверка ограничения внешнего ключа проходит успешно и исключение не возбуждается, кроме случая, когда такой отдел уже существует до вставки. Однако обработчик в секции `EXCEPTION` перехватывает эту ситуацию и позволяет операции успешно выполниться.

Примечание. Этот процесс работает в Oracle8*i* и последующих версиях. Пример, описанный на слайде, вызывает ошибку на шаге выполнения в версиях до Oracle8*i*.

Триггеры типа INSTEAD OF



Copyright © 2004, Oracle. All rights reserved.

Триггеры типа INSTEAD OF

Используйте триггеры типа INSTEAD OF для внесения изменений в представление, в которое нельзя прямо внести изменения с помощью команды DML. Эти триггеры называются INSTEAD OF, т.к. в отличие от других триггеров сервер Oracle выполняет эти триггеры вместо команд, по которым они срабатывают. Такие триггеры используются для выполнения операций INSERT, UPDATE и DELETE непосредственно в базовые таблицы. Вы можете применить к представлению команды INSERT, UPDATE и DELETE . Триггер INSTEAD OF прорабатывает незаметно как фоновая операция и выполняет необходимые действия в соответствие с этими командами. Представление не может быть обновлено с помощью обычных команд DML, если оно содержит операторы на множествами, групповые функции, такие предложения, как GROUP BY, CONNECT BY, START, оператор DISTINCT или соединения. Например, если представление содержит более одной таблицы, добавление в представление может повлечь за собой вставку в одну таблицу и изменение в другой. Поэтому вы можете создать триггер INSTEAD OF, который срабатывает при добавлении записей в представление. Вместо первоначальной операции добавления выполняется тело триггера, в результате чего производится вставка в одну таблицу и обновление другой таблицы.

Примечание: если представление может быть обновлено напрямую и имеется триггер INSTEAD OF, триггер имеет приоритет. Триггеры INSTEAD OF – это строчные триггеры. Параметр CHECK, заданный для представления, не учитывается, когда вставка или изменение представления производится с использованием триггеров INSTEAD OF. Проверка должна выполняться в теле этого триггера.

Создание триггера типа INSTEAD OF

Вставка в представление EMP_DETAILS, которое основывается на таблицах EMPLOYEES и DEPARTMENTS:

```
INSERT INTO emp_details
VALUES (9001,'ABBOTT',3000, 10, 'Administration');
```

1 Вместо вставки в EMP_DETAILS

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
100	King	90
101	Kochhar	90
102	De Haan	90

2 Вставка в NEW_EMPS

EMPLOYEE_ID	LAST_NAME	SALARY	DEPARTMENT_ID
100	King	24000	90
101	Kochhar	17000	90
102	De Haan	17000	90
...			
9001	ABBOTT	3000	10

3 Обновление NEW_DEPTS

DEPARTMENT_ID	DEPARTMENT_NAME	DEPT_SA
10	Administration	940
20	Marketing	1900
30	Purchasing	3012
40	Human Resources	650
...		

Copyright © 2004, Oracle. All rights reserved.

Создание триггера типа INSTEAD OF

Вы можете создать триггер типа INSTEAD OF для сопровождения базовых таблиц, на основе которых создано представление. В примере на слайде описывается вставка в представление EMP_DETAILS, запрос которого основывается на таблицах EMPLOYEES и DEPARTMENTS. Триггер NEW_EMP_DEPT (его тип INSTEAD OF) выполняется вместо операции INSERT, которая вызывает срабатывание этого триггера. Для базовых таблиц представления EMP_DETAILS в триггере производятся соответствующие команды INSERT и UPDATE. Поэтому вместо вставки записи о новом сотруднике в таблицу EMPLOYEES выполняются следующие действия :

1. Срабатывает триггер типа INSTEAD OF с именем NEW_EMP_DEPT.
2. Строка вставляется в таблицу NEW_EMPS.
3. Обновляется столбец DEPT_SAL таблицы NEW_DEPTS. В результате величина оклада нового сотрудника добавляется к существующему фонду оплаты его отдела.

Примечание: код этого примера приведен на следующих страницах.

Создание триггера типа INSTEAD OF

Используйте триггер типа INSTEAD OF, чтобы выполнить DML-операцию для сложного представления:

```
CREATE TABLE new_emps AS
 SELECT employee_id,last_name,salary,department_id
 FROM employees;

CREATE TABLE new_depts AS
 SELECT d.department_id,d.department_name,
        sum(e.salary) dept_sal
 FROM employees e, departments d
 WHERE e.department_id = d.department_id;

CREATE VIEW emp_details AS
 SELECT e.employee_id, e.last_name, e.salary,
        e.department_id, d.department_name
 FROM employees e, departments d
 WHERE e.department_id = d.department_id
GROUP BY d.department_id,d.department_name;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Создание триггера типа INSTEAD OF (продолжение)

В примере на слайде создаются две новые таблицы NEW_EMPS и NEW_DEPTS на основе таблиц EMPLOYEES и DEPARTMENTS соответственно. Кроме того, добавляется представление EMP_DETAILS на основе таблиц EMPLOYEES и DEPARTMENTS.

Для представления со сложной структурой запроса не всегда можно напрямую выполнить операцию DML, в результате которой данные должны попасть в базовые таблицы. В приводим примере требуется создать триггер NEW_EMP_DEPT типа INSTEAD OF, текст которого показывается на следующей странице. Триггер NEW_DEPT_EMP обрабатывает операции DML следующим образом:

- При вставке строки в EMP_DETAILS вместо прямого добавления строки в представление производятся операции вставки строк в таблицы NEW_EMPS и NEW_DEPTS с использованием значений из команды INSERT.
- При обновлении и удалении строки через представление EMP_DETAILS производятся соответствующие действия над строками таблиц NEW_EMPS и NEW_DEPTS.

Примечание: триггеры INSTEAD OF могут быть созданы только для представлений; в них разрешается использовать лишь опции времени срабатывания (BEFORE и AFTER).

Создание триггера типа `INSTEAD OF` (продолжение)

```
CREATE OR REPLACE TRIGGER new_emp_dept
INSTEAD OF INSERT OR UPDATE OR DELETE ON emp_details
FOR EACH ROW
BEGIN
  IF INSERTING THEN
    INSERT INTO new_emps
    VALUES (:NEW.employee_id, :NEW.last_name,
            :NEW.salary, :NEW.department_id);
    UPDATE new_depts
      SET dept_sal = dept_sal + :NEW.salary
      WHERE department_id = :NEW.department_id;
  ELSIF DELETING THEN
    DELETE FROM new_emps
      WHERE employee_id = :OLD.employee_id;
    UPDATE new_depts
      SET dept_sal = dept_sal - :OLD.salary
      WHERE department_id = :OLD.department_id;
  ELSIF UPDATING ('salary') THEN
    UPDATE new_emps
      SET salary = :NEW.salary
      WHERE employee_id = :OLD.employee_id;
    UPDATE new_depts
      SET dept_sal = dept_sal +
                     (:NEW.salary - :OLD.salary)
      WHERE department_id = :OLD.department_id;
  ELSIF UPDATING ('department_id') THEN
    UPDATE new_emps
      SET department_id = :NEW.department_id
      WHERE employee_id = :OLD.employee_id;
    UPDATE new_depts
      SET dept_sal = dept_sal - :OLD.salary
      WHERE department_id = :OLD.department_id;
    UPDATE new_depts
      SET dept_sal = dept_sal + :NEW.salary
      WHERE department_id = :NEW.department_id;
  END IF;
END;
/
```

Сравнение триггеров базы данных и хранимых процедур

Триггеры	Процедуры
Команда `CREATE TRIGGER`	Команда `CREATE PROCEDURE`
Исходный код хранится в представлении `USER_TRIGGERS`	Исходный код хранится в представлении `USER_SOURCE`
Вызывается неявно	Вызывается явно
Команды `COMMIT`, `SAVEPOINT` и `ROLLBACK` не разрешаются	Команды `COMMIT`, `SAVEPOINT` и `ROLLBACK` разрешаются

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Сравнение триггеров базы данных и хранимых процедур

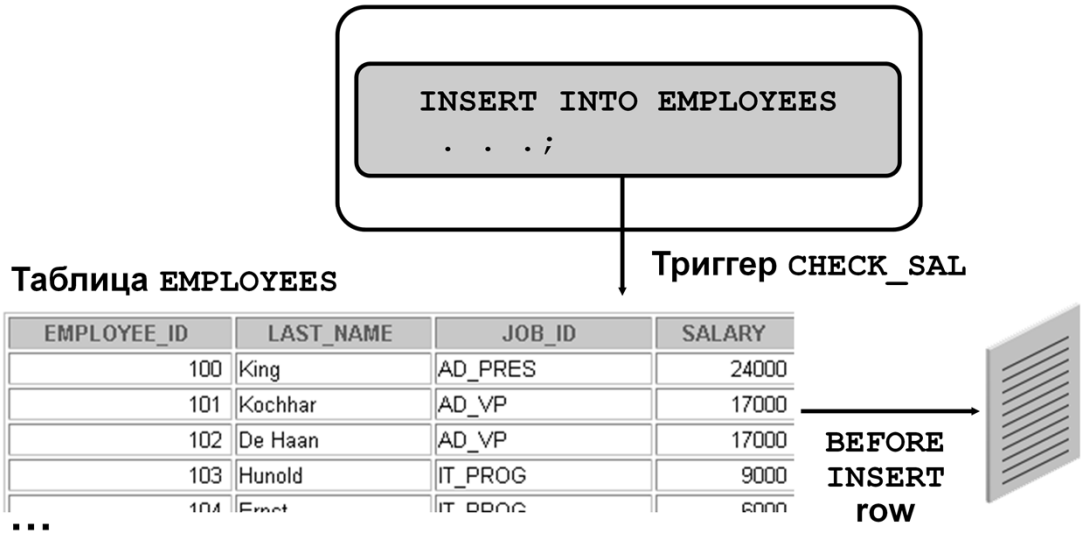
Между триггерами базы данных и процедурами имеются различия:

Триггер базы данных	Хранимая процедура
Вызывается неявно	Вызывается явно
Команды `COMMIT`, `ROLLBACK` и `SAVEPOINT` не разрешены в теле триггера. Возможно неявное выполнение фиксации или отката путем вызова процедуры, но это не рекомендуется делать из-за побочных эффектов, оказывающих влияние на транзакции.	Команды `COMMIT`, `ROLLBACK` и `SAVEPOINT` разрешены в теле процедуры.

Триггеры полностью компилируются, когда выполняется команда `CREATE TRIGGER` и `P`-код заносится в словарь данных.

Примечание: триггер создается даже в случае, если во время компиляции возникают ошибки.

Сравнение триггеров базы данных и Oracle Forms



Copyright © 2004, Oracle. All rights reserved.

Сравнение триггеров базы данных и Oracle Forms

Не путайте триггеры базы данных с триггерами Form Builder.

Триггер базы данных	Триггер Form Builder
Срабатывает вследствие действий, выполняемых в любом приложении или инструментальном средстве базы данных	Выполняется только в рамках определенного приложения Form Builder
Всегда срабатывает по командам SQL DML, DDL или после определенного действия в базе данных.	Срабатывает на событие в приложении (переход от одного поля к другому , нажатие на клавишу и т.д.)
Может быть операторным или строчным	Операторные и строчные триггеры не различаются
В случае сбоя при выполнении триггера производится откат команды, по которой сработал триггер	В случае сбоя при выполнении триггера курсор замораживается, и возможен откат всей транзакции.
Срабатывает независимо от триггеров Form Builder и в дополнение к ним	Срабатывает независимо от триггеров базы данных и в дополнение к ним
Выполняется под доменом безопасности создателя триггера	Выполняется под доменом безопасности пользователя, который работает с Form Builder

Управление триггерами

- **Отключение и повторное включение триггера базы данных:**

```
ALTER TRIGGER имя_триггера DISABLE | ENABLE
```

- **Отключение и повторное включение всех триггеров для таблицы:**

```
ALTER TABLE имя_таблицы DISABLE | ENABLE
  ALL TRIGGERS
```

- **Перекомпиляция триггера для таблицы:**

```
ALTER TRIGGER имя_триггера COMPILE
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Управление триггерами

Триггер имеет два состояния или режима: ENABLED и DISABLED. Когда триггер только создается, он включается по умолчанию. Для включенных триггеров сервер Oracle проверяет ограничения и гарантирует, что триггеры не могут нарушить никакие правила целостности. Кроме того, сервер Oracle обеспечивает согласованное чтение для запросов и ограничений, управляет зависимостями и обеспечивает двухфазную фиксацию транзакций, если триггер обновляет удаленные таблицы в распределенной базе данных.

Отключение триггера

- Отключить конкретный триггер можно с помощью команды ALTER TRIGGER; отключить *все* триггеры таблицы можно, используя команду ALTER TABLE.
- Триггер можно отключать для повышения производительности или для подавления проверок на соблюдение правил целостности больших объемов данных с помощью таких утилит, как SQL*Loader. Иногда приходится отключать триггер, если он ссылается на объект базы данных, недоступный в данный момент времени из-за ошибки соединения, отказа диска, автономного файла данных или автономного табличного пространства.

Перекомпиляция триггера

- Для явной перекомпиляции триггера, ставшего недействительным (invalid), используется команда ALTER TRIGGER.
- Если используется команда ALTER TRIGGER с параметром COMPILE, триггер перекомпилируется независимо от того, действителен он или нет.

Удаление триггеров

Для удаления триггера базы данных используется команда `DROP TRIGGER`:

```
DROP TRIGGER имя_триггера;
```

Пример:

```
DROP TRIGGER secure_emp;
```

Примечание: при удалении таблицы удаляются все ее триггеры.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Удаление триггеров

Если триггер больше не нужен, его можно удалить в *i*SQL*Plus , используя команду SQL для удаления триггера.

Проверка работы триггеров

- **Проверьте не только каждую операцию, вызывающую срабатывание триггера, но и операции, не затрагивающие триггер.**
- **Проверьте каждый случай с предложением `WHEN`.**
- **Проверьте как прямое срабатывание от базовой операции с данными, так и косвенное из процедуры.**
- **Проверьте влияние триггера на другие триггеры.**
- **Проверьте влияние других триггеров на данный триггер.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Проверка работы триггеров

Тестирование кода может потребовать значительного времени.

- Необходимо отдельно проверить несколько случаев, чтобы убедиться в правильности работы триггера:
  - сначала проверьте наиболее общие сценарии;
  - проверьте также, что при наиболее общих ошибочных условиях, производится соответствующая обработка.
- Чем сложнее триггер, тем более детальная проверка может потребоваться. Например, для строчного триггера с предложением `WHEN` необходимо убедиться в том, что триггер срабатывает при заданных условиях. Если же триггеры выполняются каскадно, необходимо протестировать влияние одного триггера на другие и убедиться в том, что достигаются желаемые результаты.
- Для отладки триггеров используйте пакет `DBMS_OUTPUT`.

Итоги

- Создание триггеров базы данных, срабатывающих при выполнении операций DML.
- Создание операторных и строчных триггеров.
- Использование правил срабатывания триггеров в базе данных.
- Включение, отключение и сопровождение триггеров базы данных.
- Разработка стратегии проверки триггеров
- Удаление триггеров базы данных.

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Итоги

На этом уроке рассматривалось создание триггеров базы данных, выполняемых до, после и вместо заданной операции DML. Триггеры связаны с таблицей или представлением базы данных. Для операций DML над таблицами применяются триггеры `BEFORE` и `AFTER`. Триггер `INSTEAD OF` позволяет заменить операции DML над представлением на соответствующие команды DML для других таблиц базы данных.

Триггеры по умолчанию включены, однако их можно выключить, запретив таким образом их выполнение до повторного включения.

При изменении бизнес-правил триггеры можно удалить или внести в них требуемые изменения.

Обзор практического занятия 10

- Создание строчных триггеров
- Создание операторного триггера
- Вызов процедур из триггера

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Обзор практического занятия 10

На этом практическом занятии вы создадите операторные и строчные триггеры. Вы также создадите процедуры, вызываемые из триггеров.

Практическое занятие 10

1. В строках таблицы JOBS хранятся допустимые минимальные и максимальные оклады для различных идентификаторов должностей (JOB_ID). Вам нужно создать код, который позволяет убедиться в том, что при выполнении операций вставки и изменения оклад сотрудника остается в диапазоне, соответствующем определенному виду должностей.
   (а). Напишите процедуру CHECK_SALARY, принимающую два параметра: строковое значение идентификатора должности и оклад. По идентификатору должности процедура должна выявить минимальный и максимальный оклады. Если параметр, содержащий оклад, не попадает в диапазон (включающий верхнюю и нижнюю границы) для переданной должности, тогда в приложении должно возбуждаться исключение, по которому возвращается сообщение: “Неверный оклад <оклад>. Оклад для должности с типом <job_id> должен быть от <min> до <max>”. Замещайте различные элементы в выдаваемом сообщении значениями передаваемых параметров и переменных, в которые по запросу выбираются данные.
   (б). Создайте триггер CHECK_SALARY_TRG для таблицы EMPLOYEES, срабатывающий перед операциями INSERT или UPDATE для каждой строки. Триггер должен вызывать процедуру CHECK_SALARY для проверки бизнес-логики. При вызове должны передаваться в качестве параметров идентификатор должности и оклад.
2. Проверьте триггер CHECK_SAL_TRG в следующих случаях:
   (а). Добавьте с помощью процедуры EMP_PKG.ADD_EMPLOYEE сотрудника Eleanor Beh в отдел 30. Что произошло и почему ?
   (б). Измените оклад сотрудника с номером 115 на $2000. В отдельной операции обновления измените идентификатор должности на HR_REP. Что произошло в каждом случае ?
   (в). Измените оклад сотрудника с номером 115 на $2800. Что произошло?
3. Измените триггер CHECK_SALARY_TRG так, чтобы он срабатывал только, когда действительно обновляются наименование должности и величина оклада.
   (а). С помощью предложения WHEN реализуйте бизнес-правило, по которому происходит проверка того, изменяются ли значения столбцов JOB_ID или SALARY.
   Примечание: используя ссылки OLD.имя_столбца, обеспечьте обработку первоначальных неопределенных значений (NULL) столбцов при выполнении операции INSERT; в противном случае операция завершится с ошибкой.
   (б). Проверьте триггер, выполнив процедуру EMP_PKG.ADD_EMPLOYEE со следующими значениями параметров: first_name='Eleanor', last name='Beh', email='EBEH', job='IT_PROG', sal=5000.

Практическое занятие 10 (продолжение)

(в). Увеличьте оклад на $2000 для сотрудников, занимающих должность `IT_PROG`. Что произошло?

(г). Установите сотруднику `Eleanor Beh` оклад, равный $9000.
Подсказка. Воспользуйтесь командой `UPDATE` с подзапросом в предложении `WHERE`. Что произошло?

(д). Измените должность сотрудника `Eleanor Beh` на `ST_MAN`, используя другую команду `UPDATE` с подзапросом. Что произошло ?

4. Необходимо запретить удаление записей о служащих в рабочее время.

(а). Создайте операторный триггер `DELETE_EMP_TRG`, который запрещает удаление строк в рабочие дни недели с 9:00 утра до 6:00 вечера.

(б). Попытайтесь удалить записи о служащих с должностью `SA_REP`, не работающих ни в каком отделе.
Примечание: это служащий `Grant` с номером 178.